

Laporan Analisis: Tugas Hybrid Socket

Vicky Mahfudy 25/573019/PPA/07207

Bagian A: Implementasi Pemrograman Soket

A.1 Layanan TCP Multi-Client

Server (TCPServer.py) membuat 1 welcoming socket yang tetap terikat ke port 12000 seumur hidup proses, hanya dipakai untuk `accept()`. Setiap `accept()` yang berhasil menghasilkan connection socket baru, yang langsung diserahkan ke thread worker terpisah (`threading.Thread`) untuk melayani seluruh pertukaran data dengan client tersebut. Welcoming socket tidak pernah dipakai untuk kirim/terima data aplikasi dan hidup selama server berjalan, sedangkan connection socket dibuat per client, hanya hidup selama sesi client itu, dan ditutup begitu client disconnect. Karena kedua jenis socket ini terpisah, main thread bisa langsung `accept()` client baru tanpa menunggu client lama selesai, itulah dasar konkurensi.

Bukti: 3 client (A, B, C) dijalankan hampir bersamaan, connect-nya saling menyusul sebelum satu pun disconnect, dan Wireshark menunjukkan tiga three-way handshake terpisah dari tiga port ephemeral berbeda, semuanya menuju port 12000 yang sama.

Potongan kode welcoming socket dan `accept()` (Kode/TCPServer.py):

```
def main():
    serverSocket = socket(AF_INET, SOCK_STREAM)
    serverSocket.setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)
    serverSocket.bind(('', SERVER_PORT))
    serverSocket.listen(5)

    print(f'TCP hybrid server ready, listening on port {SERVER_PORT}...')

    while True:
        # The welcoming socket's only job: block until a new client shows
        # up, then hand off the resulting connection socket to a worker
        # thread and immediately go back to listening for the next one.
        connectionSocket, addr = serverSocket.accept()
        clientThread = threading.Thread(
            target=handle_client,
            args=(connectionSocket, addr),
            daemon=True
        )
        clientThread.start()
```

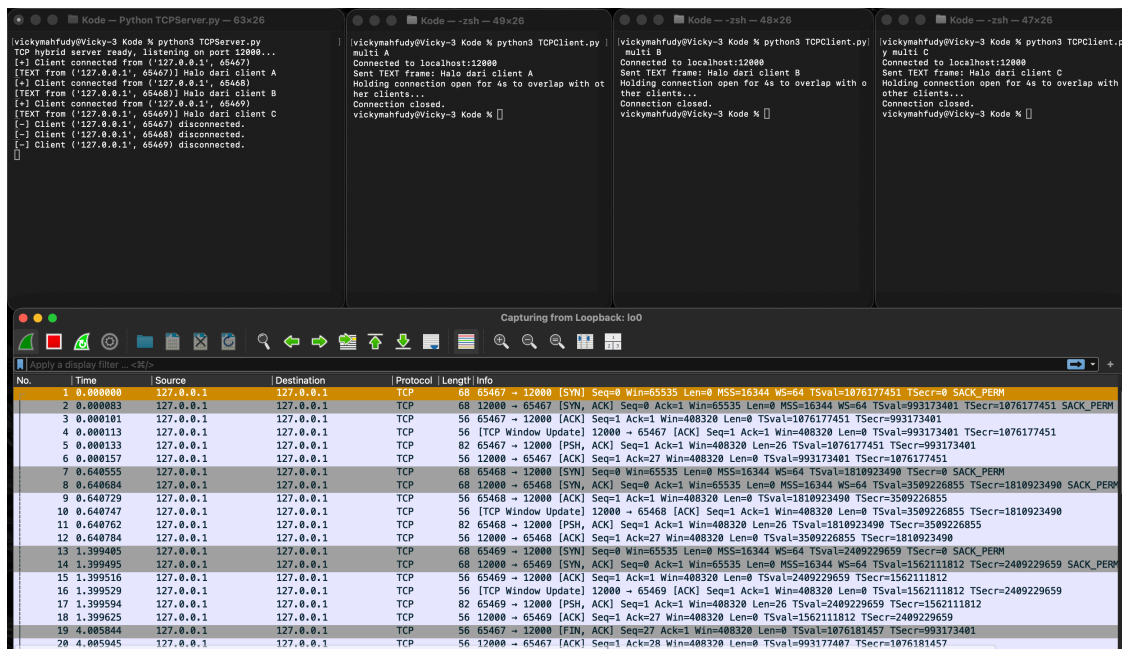


Figure 1: Bukti konkurensi TCP multi-client: 3 client terhubung bersamaan, log server dan tiga three-way handshake terpisah di Wireshark

A.2 Layanan UDP Pinger

Client (UDPPingerClient.py) memakai `clientSocket.settimeout(1.0)` sebelum mengirim 10 ping berurutan. Jika balasan tidak datang dalam 1 detik, `exception timeout` ditangkap, ping tersebut dihitung hilang, dan loop lanjut ke ping berikutnya tanpa pernah blocking selamanya. Di akhir 10 kali kirim, client menghitung RTT minimum/rata-rata/maksimum dari ping yang berhasil, serta packet loss rate. Untuk mendemonstrasikan penanganan loss secara nyata, server (UDPPingerServer.py) sengaja men-drop ~30% ping yang masuk secara acak.

Hasil uji: ping 1, 2, dan 10 di-drop server sehingga client timeout tepat pada ketiganya; ping 3-9 berhasil dibalas dengan RTT sub-milidetik. Statistik akhir: 10 transmitted, 7 received, 30% packet loss.

Potongan kode client UDP dengan `settimeout(1.0)` dan penanganan paket hilang (Kode/UDPPingerClient.py):

```
clientSocket = socket(AF_INET, SOCK_DGRAM)
```

```
# Give up waiting for a reply after 1 second: this is the client-side
# defense against unreliable packet delivery required by the brief.
clientSocket.settimeout(1.0)
```

```
NUM_PINGS = 10
rtt_list = []
lost_count = 0
```

```
for sequence_number in range(1, NUM_PINGS + 1):
    sendTime = time.time()
```

```

message = f'Ping {sequence_number} {sendTime}'

try:
    startTime = time.time()
    clientSocket.sendto(message.encode(), (serverName, serverPort))

    rttMessage, serverAddress = clientSocket.recvfrom(1024)
    endTime = time.time()

    rtt = endTime - startTime
    rtt_list.append(rtt)
    print(f'Reply from {serverAddress}: {rttMessage.decode()!r}, '
          f'RTT = {rtt:.6f} s')
except timeout:
    lost_count += 1
    print(f'Ping {sequence_number}: Request timed out (no reply within 1.0 s)')

```

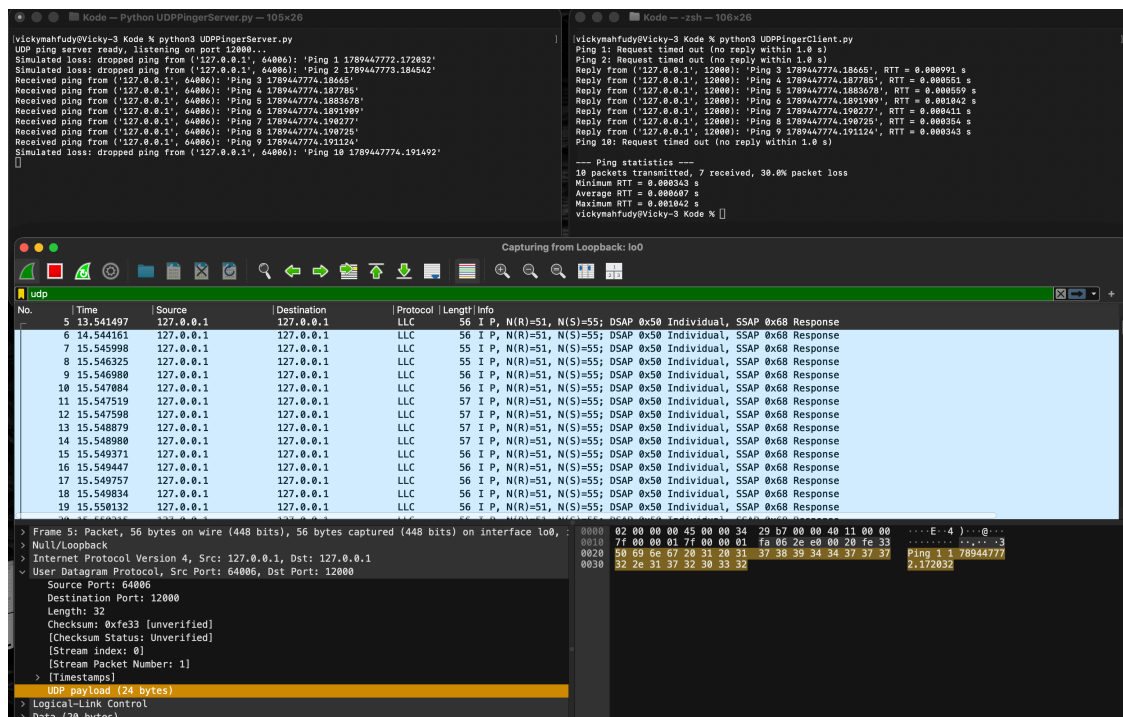


Figure 2: Bukti UDP Pinger: log server/client dan packet capture Wireshark datagram UDP request/reply pada port 12000

Bagian B: Analisis Mendalam & Arsitektur Jaringan

B.1 Byte-Stream vs Message Boundary

TCP memandang data sebagai aliran byte tanpa batas pesan: TCP boleh menggabungkan beberapa `send()` berurutan menjadi satu segmen atau memecah satu `send()` menjadi beberapa segmen, dan `recv()` tidak dijamin selaras satu-satu dengan `send()`. UDP sebaliknya menjaga

message boundary: setiap `sendto()` = satu datagram utuh, setiap `recvfrom()` mengembalikan tepat satu datagram itu, karena UDP tidak punya abstraksi byte stream.

Dampak teknis: `TCPClient.py demo3` mengirim 3 pesan tanpa delay. Wireshark membuktikan hanya **2 segmen TCP** yang benar-benar terkirim: segmen pertama `Len=21` (pesan pertama), segmen kedua `Len=39` yaitu gabungan pesan kedua (19 byte) dan ketiga (20 byte) yang digabung TCP menjadi satu segmen karena dikirim tanpa delay.

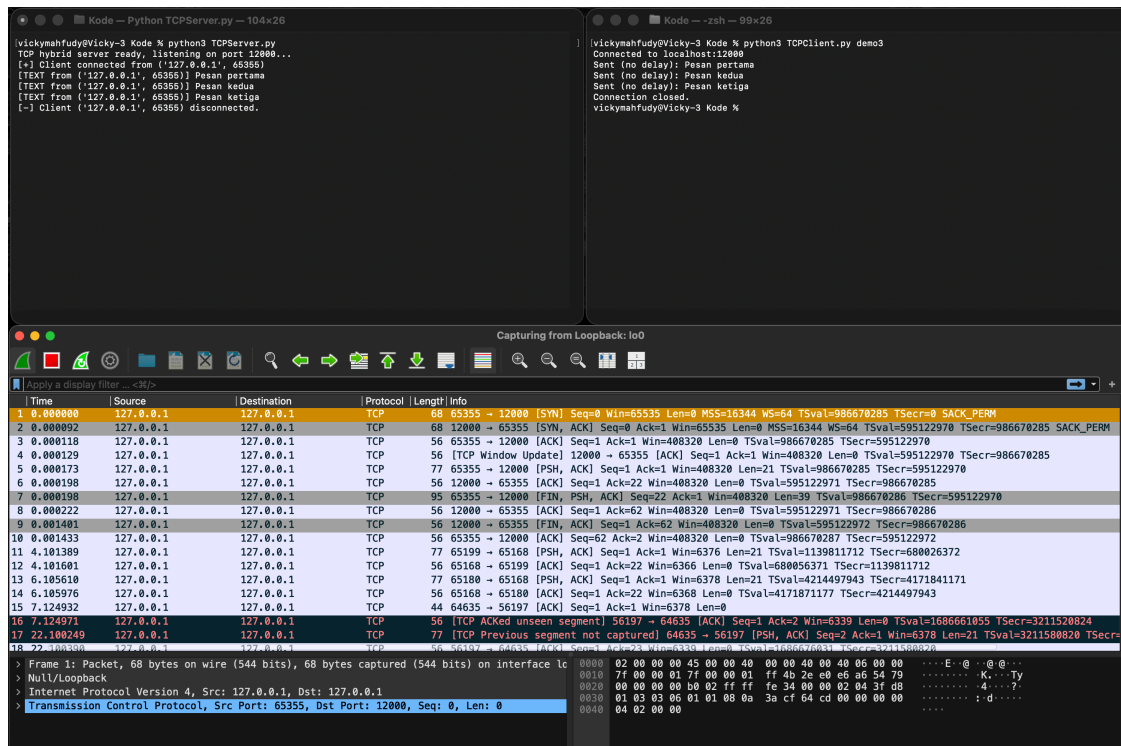


Figure 3: Bukti Wireshark: 3 pesan dikirim tanpa delay, hanya 2 segmen TCP terkirim di kabel

Solusi protokol aplikasi: setiap frame diawali 4-byte length prefix yang memberitahu penerima persis berapa byte payload yang harus dibaca. Fungsi `recv_exact()` terus membaca sampai jumlah byte lengkap, sehingga pemisahan pesan dilakukan di level aplikasi, bukan mengandalkan batas `recv()`. Terbukti server tetap menampilkan 3 pesan terpisah dengan benar meski di kabel hanya 2 segmen.

B.2 Analisis Skalabilitas Socket

Server TCP membuka **N + 1** socket untuk N client: 1 welcoming socket tetap (port 12000) plus N connection socket (satu per client aktif, dibedakan lewat 4-tuple IP/port lokal-remote). Terbukti pada demo multi-client: 3 client = 3 connection socket + 1 welcoming socket = 4 socket total.

Server UDP hanya butuh **1 socket** karena connectionless, tidak ada `listen()/accept()/state` per sesi; `recvfrom()` sekaligus memberi alamat pengirim untuk dibalas langsung. Implikasi: server UDP memakai 1 FD dan 1 buffer kernel berapa pun banyak client, sedangkan server TCP memakai FD dan buffer yang bertumbuh $O(N)$, dan tiap connection socket mengonsumsi entry tabel koneksi

kernel yang membatasi jumlah koneksi simultan secara praktis. Server UDP tidak punya batasan setara ini.

B.3 Evolusi Transport & QUIC

QUIC berjalan di atas UDP karena: (1) **deployability** — middlebox internet (NAT, firewall) hanya meloloskan TCP/UDP, protokol IP baru akan banyak di-drop; (2) **menghindari kekakuan kernel OS** — TCP ada di kernel dan sulit diubah cepat, sedangkan di atas UDP QUIC bisa mengimplementasikan seluruh logic transport sendiri di user-space dan berevolusi lewat update aplikasi/library; (3) **kontrol penuh** — UDP tidak memaksakan model reliability/ordering, sehingga QUIC bebas merancang multiplexing sesuai kebutuhan modern.

Solusi HOL blocking: TCP hanya mengenal satu byte stream per koneksi, sehingga satu segmen hilang memblokir seluruh stream setelahnya (termasuk data resource lain yang di-multiplex, seperti HTTP/2 di atas TCP). QUIC mengimplementasikan multiplexing di level stream native dengan buffer reliability independen per stream: jika satu paket stream A hilang, hanya stream A yang menunggu retransmisi, stream lain yang sudah lengkap tetap langsung diserahkan ke aplikasi.

B.4 Skenario Bind Explicit pada Client

Default-nya OS memilih port ephemeral otomatis saat client pertama mengirim data. `clientSocket.bind('', 5432)` eksplisit memaksa socket memakai port 5432 tetap pada semua interface lokal, bukan port ephemeral acak.

Apakah server masih bisa membalas: Ya, karena balasan UDP murni bergantung pada alamat pengirim yang tercatat di header paket yang diterima lewat `recvfrom()`; server tinggal `sendto()` balik ke alamat itu, berfungsi identik dengan port ephemeral biasa.

Potensi kendala 2 instansiasi bersamaan: Port 5432 bersifat eksklusif per host; jika instansiasi client kedua juga mencoba bind ke port yang sama, `bind()` akan gagal dengan `OSError: Address already in use`, karena port sudah dipakai instansiasi pertama. Berbeda dari implicit bind, di mana setiap instansiasi otomatis mendapat port ephemeral berbeda dari OS sehingga banyak instansiasi bisa berjalan bersamaan tanpa konflik.